

Merge Sort, Quick Sort, Limite Inferior e Counting Sort

Prof. Dr. André Yoshiaki Kashiwabara

DCOMP — Universidade Federal de Sergipe

Merge Sort

- Insertion Sort: $\Theta(n^2)$ no pior caso
- Podemos fazer melhor?
 - **Sim!** Usando *divisão e conquista*
- **Merge Sort:** $\Theta(n \log n)$ em *todos* os casos
- Algoritmo *estável* e *paralelizável*
- Base para variantes externas (External Merge Sort)

Algoritmo	Pior caso
Insertion Sort	$\Theta(n^2)$
Merge Sort	$\Theta(n \log n)$
Quick Sort	$\Theta(n^2)$

Três etapas

1. **Dividir:** quebrar o problema em subproblemas menores
2. **Conquistar:** resolver recursivamente (ou diretamente se trivial)
3. **Combinar:** juntar as soluções parciais

No Merge Sort

- **Dividir:** partir $A[p \dots r]$ ao meio em $A[p \dots q]$ e $A[q+1 \dots r]$
- **Conquistar:** ordenar recursivamente cada metade
- **Combinar:** intercalar (*merge*) as duas metades ordenadas

Algorithm 1 MERGESORT(A, p, r)

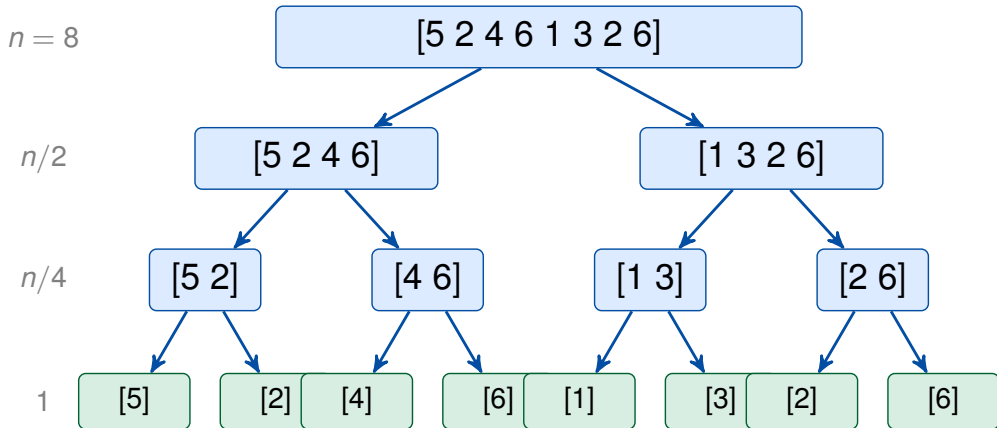
```
1: if  $p < r$  then  
2:    $q \leftarrow \lfloor (p + r) / 2 \rfloor$   
3:   MERGESORT( $A, p, q$ )  
4:   MERGESORT( $A, q + 1, r$ )  
5:   MERGE( $A, p, q, r$ )  
6: end if
```

- Chamada inicial: MERGESORT($A, 1, n$)
- Caso base: subarray de tamanho ≤ 1 já está ordenado
- O *trabalho real* acontece na etapa **Combine**: MERGE

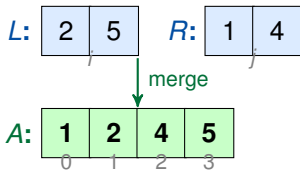
Algorithm 2 MERGE(A, p, q, r)

```
1: Copiar  $A[p..q]$  para  $L[1..|L|]$  e  $A[q+1..r]$  para  $R[1..|R|]$ 
2: Adicionar sentinela  $\infty$  ao final de  $L$  e  $R$ 
3:  $i \leftarrow 1$ ;  $j \leftarrow 1$ 
4: for  $k \leftarrow p$  to  $r$  do
5:   if  $L[i] \leq R[j]$  then
6:      $A[k] \leftarrow L[i]$ ;  $i \leftarrow i + 1$ 
7:   else
8:      $A[k] \leftarrow R[j]$ ;  $j \leftarrow j + 1$ 
9:   end if
10: end for
```

- Complexidade de MERGE: $\Theta(n)$ — percorre $r - p + 1$ posições
- Sentinela ∞ simplifica o código (evita checar fim de lista)



Intercalando $L = [2, 5]$ e $R = [1, 4]$ em A :



- A cada passo: comparar $L[i]$ e $R[j]$, copiar o menor para A
- Total: $r - p + 1$ comparações/cópias $\Rightarrow \Theta(n)$

Equação de recorrência

$$T(n) = \begin{cases} \Theta(1) & \text{se } n = 1 \\ 2T\left(\frac{n}{2}\right) + \Theta(n) & \text{se } n > 1 \end{cases}$$

Interpretação:

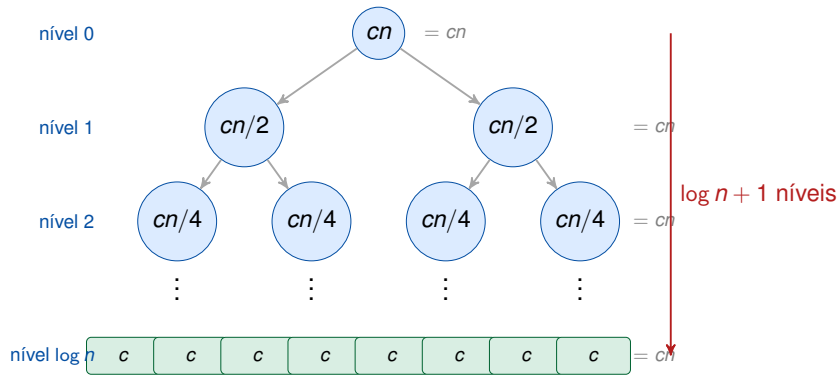
- $2T(n/2)$: duas chamadas recursivas de tamanho $n/2$
- $\Theta(n)$: custo do MERGE

Solução:

$$T(n) = \Theta(n \log n)$$

- Pelo *Teorema Mestre* (caso 2):
 $a = 2, b = 2, f(n) = \Theta(n)$
 $n^{\log_b a} = n^1 \Rightarrow \text{caso 2}$

Análise pela Árvore de Recursão



$$T(n) = cn \cdot (\log n + 1) = \Theta(n \log n)$$

Merge Sort — todos os casos são iguais

Caso	Divisão	Merge	Total
Melhor	$\Theta(\log n)$	$\Theta(n)$ por nível	$\Theta(n \log n)$
Médio	$\Theta(\log n)$	$\Theta(n)$ por nível	$\Theta(n \log n)$
Pior	$\Theta(\log n)$	$\Theta(n)$ por nível	$\Theta(n \log n)$

Desvantagem

Espaço auxiliar: $\Theta(n)$ — não é *in-place*
(as cópias L e R no MERGE exigem memória extra)

Propriedade	Insertion Sort	Merge Sort
Pior caso	$\Theta(n^2)$	$\Theta(n \log n)$
Melhor caso	$\Theta(n)$	$\Theta(n \log n)$
Estável	Sim	Sim
In-place	Sim	Não
Uso prático	n pequeno	n grande

Quick Sort

Divisão e Conquista (ao contrário)

1. **Escolher** um elemento *pivô* x
2. **Particionar**: rearranjar A em
 $\underbrace{[\leq x]}_{\text{esq}} \quad [x] \quad \underbrace{[\geq x]}_{\text{dir}}$
3. **Conquistar**: ordenar recursivamente esquerda e direita
4. **Combinar**: nada a fazer!

Características

- **In-place**: $O(\log n)$ pilha
- Caso médio: $\Theta(n \log n)$
- Pior caso: $\Theta(n^2)$
- Na prática: muito eficiente

Algorithm 3 QUICKSORT(A, p, r)

```
1: if  $p < r$  then  
2:    $q \leftarrow \text{PARTITION}(A, p, r)$   
3:   QUICKSORT( $A, p, q - 1$ )  
4:   QUICKSORT( $A, q + 1, r$ )  
5: end if
```

- O pivô termina na posição *correta* após PARTITION
- Chamada inicial: QUICKSORT($A, 1, n$)
- Todo o trabalho está em PARTITION — $\Theta(n)$

Algorithm 4 PARTITION(A, p, r)

```
1:  $x \leftarrow A[r]$  ▷ pivô = último
2:  $i \leftarrow p - 1$ 
3: for  $j \leftarrow p$  to  $r - 1$  do
4:   if  $A[j] \leq x$  then
5:      $i \leftarrow i + 1$ 
6:     trocar  $A[i] \leftrightarrow A[j]$ 
7:   end if
8: end for
9: trocar  $A[i+1] \leftrightarrow A[r]$ 
10: return  $i + 1$ 
```

Invariante do loop

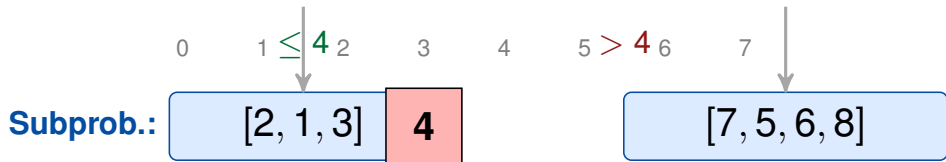
Ao início de cada iteração j :

- $A[p \dots i] \leq x$
- $A[i+1 \dots j-1] > x$
- $A[r] = x$ (pivô)

Complexidade: $\Theta(r - p)$

Visualização da Partição (Lomuto)

Particionando $A = [2, 8, 7, 1, 3, 5, 6, 4]$, pivô = 4



Quando ocorre?

Pivô é sempre o **menor** ou **maior** elemento:

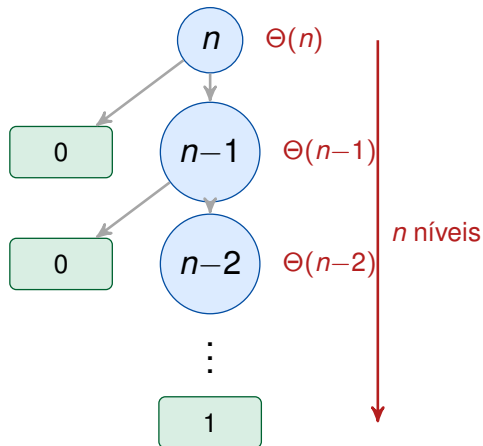
partição desequilibrada $[] \mid x \mid [n-1]$

Recorrência:

$$T(n) = T(0) + T(n-1) + \Theta(n)$$

$$T(n) = \Theta(n^2)$$

Ocorre para entrada já ordenada
com pivô = último elemento.



Melhor caso

Partição sempre **equilibrada** ($n/2 \mid n/2$):

$$T(n) = 2T(n/2) + \Theta(n) = \Theta(n \log n)$$

(mesma recorrência do Merge Sort)

Caso médio — intuição

Se partições são 9 : 1 (“ruim”):

$$T(n) = T(9n/10) + T(n/10) + \Theta(n)$$

Mesmo assim: $\Theta(n \log n)$!

Resultado principal (CLRS Teorema 7.4)

Assumindo todas as permutações igualmente prováveis, o tempo **esperado** do Quick Sort é $\Theta(n \log n)$. O número esperado de comparações é $\approx 1,386 n \log n$.

Algorithm 5 RANDPARTITION(A, p, r)

```
1:  $i \leftarrow \text{RANDOM}(p, r)$ 
2: trocar  $A[i] \leftrightarrow A[r]$ 
3: return PARTITION( $A, p, r$ )
```

Algorithm 6 RANDQUICKSORT(A, p, r)

```
1: if  $p < r$  then
2:    $q \leftarrow \text{RANDPARTITION}(A, p, r)$ 
3:   RANDQUICKSORT( $A, p, q-1$ )
4:   RANDQUICKSORT( $A, q+1, r$ )
5: end if
```

Por que randomizar?

- Elimina dependência da entrada
- Pior caso ainda $\Theta(n^2)$, mas com probabilidade *exponencialmente pequena*
- Tempo esperado: $\Theta(n \log n)$ para **qualquer** entrada

Na prática

Alternativas ao pivô aleatório:

Mediana de 3: $A[p]$, $A[\lfloor (p+r)/2 \rfloor]$, $A[r]$

Propriedade	Merge Sort	Quick Sort
Pior caso	$\Theta(n \log n)$	$\Theta(n^2)$
Caso médio	$\Theta(n \log n)$	$\Theta(n \log n)$
In-place	Não	Sim
Estável	Sim	Não
Memória extra	$\Theta(n)$	$O(\log n)$

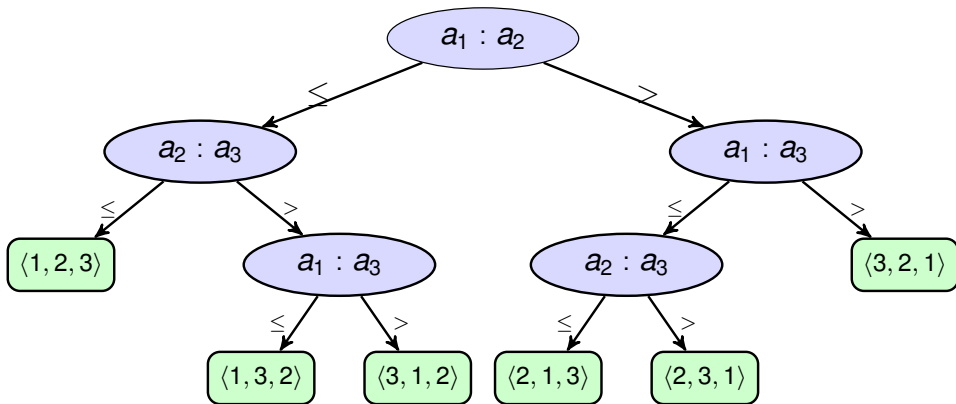
Limite Inferior

Definição

Uma **árvore de decisão** modela o comportamento de qualquer algoritmo de ordenação por comparação sobre uma entrada de tamanho n .

- Cada **nó interno** representa uma comparação $a_i : a_j$
- O resultado da comparação ($\leq$ ou $>$) determina o ramo seguido
- Cada **folha** corresponde a uma permutação final (ordenação) possível
- A **profundidade** de uma folha = número de comparações naquele caminho
- O **pior caso** = **altura** da árvore

Exemplo: Árvore de Decisão para $n = 3$



Altura = 3; todas as $3! = 6$ permutações aparecem como folhas.

Por que a Árvore Precisa de Pelo Menos $n!$ Folhas?



Argumento de Corretude

Um algoritmo de ordenação correto deve ser capaz de produzir **qualquer** uma das $n!$ permutações possíveis da entrada.

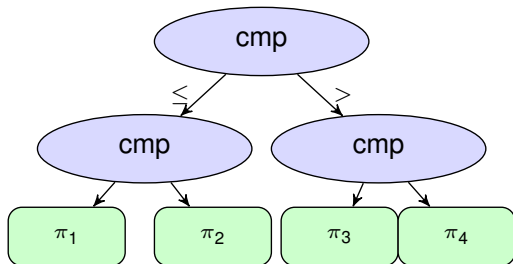
- Para n elementos, existem $n! = n \times (n-1) \times \dots \times 1$ ordenações possíveis
- Se dois inputs distintos levam à mesma folha $\Rightarrow$ o algoritmo produz a mesma saída para ambos $\Rightarrow$ **incorreto** para ao menos um deles
- Portanto, cada permutação deve corresponder a **ao menos uma folha distinta**

Conclusão

número de folhas $\geq n!$

Árvore binária de altura h :

- Tem no máximo 2^h folhas
- Cada folha = 1 resposta possível



Restrição de corretude:

$$2^h \geq n!$$

n	$n!$	$h_{\min}$
3	6	3
4	24	5
5	120	7
8	40320	16

Teorema

Qualquer árvore de decisão que ordena n elementos tem altura $h \geq \lceil \log_2(n!) \rceil$.

Prova:

1. Uma árvore binária de altura h possui no máximo 2^h folhas
2. A árvore precisa de pelo menos $n!$ folhas (slide anterior)
3. Portanto:

$$2^h \geq n! \implies h \geq \log_2(n!)$$

4. Como h é inteiro: $h \geq \lceil \log_2(n!) \rceil$ ■

Interpretação

O **pior caso** de qualquer algoritmo de ordenação por comparação requer pelo menos $\lceil \log_2(n!) \rceil$ comparações.

Fórmula de Stirling

$$n! \approx \sqrt{2\pi n} \left(\frac{n}{e}\right)^n \quad (\text{para } n \text{ grande})$$

Aplicando $\log_2$:

$$\log_2(n!) \approx \frac{1}{2} \log_2(2\pi n) + n \log_2 n - n \log_2 e$$

O termo dominante é $n \log_2 n$, então:

$$\log_2(n!) = \Theta(n \log n)$$

Prova direta simples (lower bound)

$$\log_2(n!) = \sum_{k=1}^n \log_2 k \geq \sum_{k=\lceil n/2 \rceil}^n \log_2 k \geq \frac{n}{2} \log_2 \left(\frac{n}{2}\right) = \Omega(n \log n)$$

Visualização: Crescimento de $\log_2(n!)$

comparações

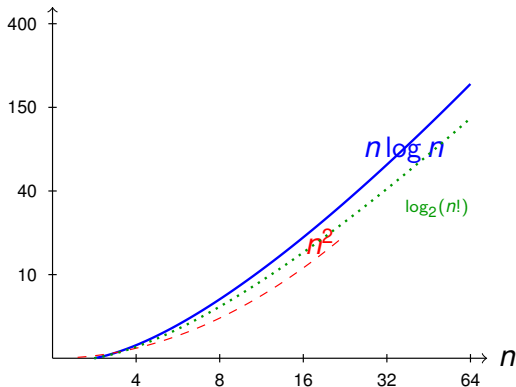


Tabela de $\lfloor \log_2(n!) \rfloor$:

n	$n!$	$\lfloor \log_2(n!) \rfloor$
4	24	4
8	40.320	15
16	$\approx 2 \times 10^{13}$	44
32	$\approx 2 \times 10^{35}$	117

Cresce como $\Theta(n \log n)$!

Teorema (CLRS, Cap. 8)

Qualquer algoritmo de ordenação por comparação requer $\Omega(n \log n)$ comparações no pior caso.

Resumo da prova:

1. Todo algoritmo de ordenação por comparação pode ser modelado por uma árvore de decisão
2. A árvore deve ter $\geq n!$ folhas (corretude)
3. Uma árvore binária com $\geq n!$ folhas tem altura $h \geq \log_2(n!)$
4. Pela aproximação de Stirling: $\log_2(n!) = \Omega(n \log n)$
5. Logo $h = \Omega(n \log n)$ ■

Corolário

HeapSort (visto em aula anterior) e MergeSort são **ótimos assintoticamente**: ambos rodam em $\Theta(n \log n)$.

Counting Sort

Limite inferior para ordenação por comparação

Qualquer algoritmo baseado em comparações precisa de $\Omega(n \log n)$ comparações no pior caso.

- Se soubermos **mais** sobre os dados, podemos ordenar mais rápido.
- **Counting Sort** não usa comparações entre elementos.
- Assume que os elementos são inteiros no intervalo $[0, k]$.
- Complexidade: $\Theta(n + k)$ — linear quando $k = O(n)$.

Quando usar?

Chaves inteiras em intervalo pequeno: idades, notas (0–100), caracteres ASCII, dígitos decimais.

✓ Favorável

- Chaves inteiras não-negativas
- Intervalo $k = O(n)$
- Necessidade de estabilidade
- Sub-rotina do Radix Sort

× Desfavorável

- Chaves reais ou strings
- $k \gg n$ (intervalo enorme)
- Memória muito restrita ($\Theta(n + k)$)
- Dados com distribuição desconhecida

$$k = O(n) \checkmark$$

$$k = O(n \log n) ?$$

$$k \gg n \times$$

COUNTINGSORT($A[1..n]$, $B[1..n]$, k)

Entrada: $A[i] \in [0, k]$; *Saída:* B ordenado

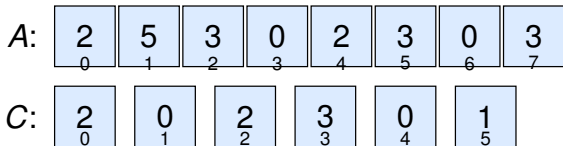
```
1 // Passo 1: contar ocorrências  $\Theta(k) + \Theta(n)$ 
2  $C[0..k] \leftarrow 0$ 
3 for  $j = 1$  to  $n$  do
4      $C[A[j]] += 1$ 
5
6 // Passo 2: acumular prefixo  $\Theta(k)$ 
7 for  $i = 1$  to  $k$  do
8      $C[i] += C[i-1]$ 
9
10 // Passo 3: espalhar (traz p/ frente)  $\Theta(n)$ 
11 for  $j = n$  downto  $1$  do
12      $B[C[A[j]]] \leftarrow A[j]$ 
13      $C[A[j]] -= 1$ 
```

Três varreduras: contar $\rightarrow$ acumular $\rightarrow$ espalhar.

Visualização: Array de Contagem (Passo 1 e 2)

Entrada $A = [2, 5, 3, 0, 2, 3, 0, 3]$ ($n = 8, k = 5$)

Passo 1 — Contar:



Passo 2 — Acumular prefixo ($C'[i] = |\{j : A[j] \leq i\}|$):



Passo 3 — Saída B (espalhar de trás para frente):



Invariante (início de cada iteração do 3.º for)

Para cada valor $v \in [0, k]$, $C[v]$ contém o número de elementos de $A[1..j]$ menores ou iguais a v que **ainda não foram colocados** em B . Equivalentemente, $B[C[v] + 1..(\text{posição original de } v)]$ está corretamente preenchido com os $n - j$ elementos já processados.

Inicialização ($j = n$):

$C[v]$ = número de elementos $\leq v$ em A inteiro. Correto pois nenhum elemento foi colocado em B ainda.

Manutenção:

Ao processar $A[j]$: colocamos $A[j]$ em $B[C[A[j]]]$ e decrementamos $C[A[j]]$, mantendo o invariante para a próxima iteração.

Término

Quando $j = 0$, todo $A[1..n]$ foi processado e B está ordenado.

Contando operações

Etapa	Operação	Custo
Inicializar C	$k + 1$ atribuições	$\Theta(k)$
Contar (1.º for)	n incrementos	$\Theta(n)$
Acumular (2.º for)	k somas	$\Theta(k)$
Espalhar (3.º for)	n escritas	$\Theta(n)$
Total		$\Theta(n + k)$

- **Tempo:** $\Theta(n + k)$
- **Espaço:** $\Theta(n + k)$ (arrays B e C)
- Se $k = O(n)$: tempo **linear** $\Theta(n)$
- Não é baseado em comparação $\Rightarrow$ não viola o limite $\Omega(n \log n)$

$\Theta(k)$ — init C

$\Theta(n)$ 1.º for

Definição — Algoritmo estável

Um algoritmo de ordenação é **estável** se elementos com chaves iguais aparecem na saída na **mesma ordem relativa** em que aparecem na entrada.

Por que o Counting Sort é estável?

- O 3.º **for** percorre *A* **de trás para frente** ($j = n$ **downto** 1).
- $C[v]$ fornece a posição do *último* elemento não colocado com chave v .
- Elementos com a mesma chave são inseridos em *B* da direita para a esquerda, preservando a ordem original entre eles.

Importância da estabilidade

É a propriedade que torna o Counting Sort útil como sub-rotina do **Radix Sort**: dígitos menos significativos processados primeiro têm sua ordem preservada nas rodadas seguintes.

Resumo Geral

Algoritmo	Pior caso	Caso Médio	Espaço	Estável?
Insertion Sort	$\Theta(n^2)$	$\Theta(n^2)$	$O(1)$	Sim
Merge Sort	$\Theta(n \log n)$	$\Theta(n \log n)$	$\Theta(n)$	Sim
Quick Sort	$\Theta(n^2)$	$\Theta(n \log n)$	$O(\log n)$	Não
Heap Sort	$\Theta(n \log n)$	$\Theta(n \log n)$	$O(1)$	Não
Counting Sort	$\Theta(n + k)$	$\Theta(n + k)$	$O(n + k)$	Sim

- **Limite Inferior:** $\Omega(n \log n)$ para qualquer ordenação por comparação.
- **Além do limite:** Algoritmos como Counting Sort, Radix Sort e Bucket Sort exploram propriedades dos dados (ex: chaves inteiras).

-  T. Cormen, C. Leiserson, R. Rivest, C. Stein.
Introduction to Algorithms, 4.^a ed., MIT Press, 2022.
-  R. Sedgewick, K. Wayne.
Algorithms, 4.^a ed., Addison-Wesley, 2011.
-  D. Knuth.
The Art of Computer Programming, vol. 3: Sorting and Searching.
Addison-Wesley, 1998.